

EXACT FACTORISATION AND FAST COMPUTATION OF INVERTIBLE CONSTANT-Q TRANSFORMS

Facundo Franchino¹ 

Eloi Moliner² 

Vesa Välimäki² 

¹Massachusetts Institute of Technology, Cambridge, MA, USA

²Acoustics Lab, Dept. of Information and Communications Eng., Aalto University, Espoo, Finland

ABSTRACT

The constant- Q transform (CQT) represents audio on a logarithmic frequency axis. Its nonstationary Gabor formulation is exactly invertible, but the unequal numbers of time coefficients in its bands complicate GPU computation. An exact factorisation combines spectral selection, conjugation, windowing, and reordering into a fixed map between one packed Fourier transform and the shorter band inverse transforms. The factors give waveform reconstruction, real adjoints for backpropagation, and bounds on arithmetic depth and block width; overlapping slices permit streaming with bounded memory. Tests on two GPU models show that Flash-CQT reduces analysis–synthesis round-trip time by factors of two to eight relative to a baseline computing the same CQT. The proposed implementation also uses over 30% less peak temporary workspace and reaches a negligible reconstruction error, with a signal-to-noise ratio of about 130 dB, in single-precision floating-point arithmetic. These advances make Flash-CQT a practical, computationally efficient front end for spectral analysis and modern audio machine-learning systems.

Index Terms— Audio systems, fast Fourier transforms, Gabor frames, GPU computation, spectral analysis

1. INTRODUCTION

Audio analysis often begins with a time-frequency representation [1, 2]. The widely used short-time Fourier transform (STFT) analyses every frequency band with the same window, giving uniform resolution across the spectrum [1, 3]. Equal musical intervals, however, correspond to equal frequency ratios, which motivates geometrically spaced analysis bands. The constant- Q transform (CQT) uses this spacing and scales each window inversely with its centre frequency, keeping the ratio Q of centre frequency to bandwidth constant [4]. Long windows distinguish nearby low-frequency components, while short windows resolve rapid changes at high frequencies. On the CQT frequency axis, equal pitch intervals span equal distances; its narrower low-frequency bands also reflect a broad feature of auditory frequency selectivity [5]. These properties make it useful for pitch estimation, transcription and music analysis [6, 7].

Although often used for analysis alone, the CQT can also be made exactly invertible. Its synthesis operation then maps coefficients, whether analysed, edited, or generated, back to a waveform. The nonstationary Gabor transform (NSGT) provides the construction for such invertible transforms with frequency-dependent windows [8]. Velasco *et al.* used it to obtain an exactly invertible CQT [9], whereas Schörkhuber and Klapuri achieved reconstruction by other means [7]; Holighaus *et al.* later derived a sliced form for streaming [10]. Analysis and synthesis are linear and therefore differentiable, allowing gradients to pass through either operation during training. In software,

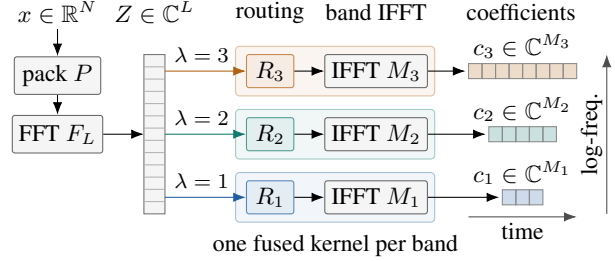


Fig. 1. Diagram of the proposed FlashCQT implementation, with three schematic bands indexed by λ . Packing P and a length- L FFT give Z , with $L = N/2$. Each shaded kernel weights the band’s bins via R_λ and gives c_λ by a length- M_λ inverse FFT.

nnAudio provides differentiable CQT analysis without an exact inverse [11], while other implementations provide both invertibility and differentiation [12, 13]. Their repeated use in training and inference nevertheless makes computational cost a concern.

The NSGT implementation [8, 9] computes an invertible CQT with one long fast Fourier transform (FFT), selection and windowing of overlapping bands in the frequency domain, and a short inverse FFT of band-dependent length for each band. On GPUs, separate calls for these operations add kernel launches, temporary buffers and data movement between stages, while zero-padding bands to a common length wastes arithmetic and storage.

Trading arithmetic and layout against data movement is a long-standing theme in Fourier computation. FFT frameworks organise matrix factors around permutations and memory access [14–19]. Edelman *et al.* [20] traded arithmetic for less communication in approximate distributed Fourier transforms. Related concerns motivate GPU methods for structured products [21] and fused operations [22, 23]; cuFFTDx permits FFTs inside custom CUDA kernels [24].

We propose a factorisation that combines the operations between the Fourier stages into a fixed map. Pairing consecutive real samples as real and imaginary parts gives a complex sequence of half the length, whose FFT retains all the information needed to recover the signal’s Fourier coefficients. For each coefficient, a precomputed table identifies the FFT outputs to combine and includes the band’s spectral window in their weights. Theorem 1 bounds the depth and width of this map, which lets the routing and the band’s short inverse FFT run in one GPU kernel, avoiding a write and read of the intermediate band spectrum (Fig. 1). Each band keeps its own length, and the factorisation gives reconstruction and real adjoints for backpropagation. We call the resulting CUDA implementation Flash-CQT.

The rest of the paper is organised as follows. Sec. 2 describes the invertible CQT in its default form. Sec. 3 derives the packed factorisation, its depth and width bounds, and derives the adjoints for

training. Sec. 4 details the CUDA implementation. Sec. 5 reports our evaluation, and Sec. 6 concludes the paper.

2. THE INVERTIBLE TRANSFORM

Let $F_n \in \mathbb{C}^{n \times n}$ be the unnormalised discrete Fourier transform (DFT) matrix [15], with row k indexing frequency bins and column j indexing samples, both from 0 to $n - 1$. With $i^2 = -1$ and H denoting conjugate transpose, its entries and inverse are

$$[F_n]_{kj} = e^{-2\pi i k j / n}, \quad F_n^{-1} = \frac{1}{n} F_n^H. \quad (1)$$

The factor $1/n$ normalises the inverse. An FFT evaluates the product with F_n without forming this matrix. We use superscript $\top$ for transpose and an overbar for complex conjugation.

The CQT, seen as a finite NSGT, decomposes a real signal $x \in \mathbb{R}^N$ of even length N into overlapping frequency bands indexed by λ , each carrying its own coefficient vector $c_\lambda \in \mathbb{C}^{M_\lambda}$ of length M_λ [8]. In the constant- Q case, bandwidth grows with frequency [9], so higher bands require more time coefficients over a fixed duration. The resulting arrays are *ragged*, with several distinct lengths M_λ .

Such transforms and their inverses can be defined through the DFT and computed with the FFT [9, 10]. The analysis operator $T: x \mapsto \{c_\lambda\}_\lambda$ maps the input signal to the coefficients of all bands, where the operator $T_\lambda: x \mapsto c_\lambda$ of band λ is

$$c_\lambda = T_\lambda x = F_{M_\lambda}^{-1} u_\lambda, \quad (2)$$

$$u_\lambda = E_\lambda \text{diag}(g_\lambda) F_N x. \quad (3)$$

Analysis first computes the spectrum $X = F_N x$ of the signal with a single DFT of length N . Band λ is specified by a real spectral window $g_\lambda \in \mathbb{R}^N$, applied by entrywise multiplication $\text{diag}(g_\lambda)$, and a selection matrix $E_\lambda \in \mathbb{R}^{M_\lambda \times N}$. The selection matrix places the bins where g_λ is nonzero into their prescribed local frequency slots, leaving unused slots zero and sending no two bins to the same slot. The resulting band weighted spectrum u_λ is mapped by the short inverse DFT $F_{M_\lambda}^{-1}$ to the band's time coefficients c_λ . Besides the positive-frequency bands, the transform retains two sidebands at zero frequency (DC) and half the sample rate (Nyquist), while negative-frequency bands follow by conjugate symmetry.

We restrict throughout to the *painless* case of frame theory [8, 25], where band supports fit their local spectra without collisions and together cover every Fourier bin. Reconstruction (synthesis) then requires only diagonal weighting. Let $g_{\lambda,k}$ denote the k th entry of g_λ . Because the windows overlap, bin k receives contributions from several bands, which synthesis must compensate. For $0 \leq k \leq N/2$, the aggregate window weight at bin k is $a_k = \sum_\lambda M_\lambda g_{\lambda,k}^2$, with the sum taken over the retained bands, and the dual weights are defined as the normalised windows $h_{\lambda,k} = M_\lambda g_{\lambda,k} / a_k$. The windows must cover every bin, so that $a_k > 0$, and the factor M_λ accounts for the normalisation of the band inverse DFT in (2). By construction, $\sum_\lambda h_{\lambda,k} g_{\lambda,k} = 1$ at every bin.

Synthesis reverses the steps of analysis. The synthesis operator $S: \{c_\lambda\}_\lambda \mapsto \hat{x}$ maps the coefficients c_λ of each band back to its spectrum with the short DFT F_{M_λ} , returns the bins to their original positions with $E_\lambda^\top$, weights them with the dual window h_λ , and applies the inverse DFT F_N^{-1} to the sum over all bands,

$$\hat{x} = S(\{c_\lambda\}_\lambda) = F_N^{-1} \sum_\lambda \text{diag}(h_\lambda) E_\lambda^\top F_{M_\lambda} c_\lambda. \quad (4)$$

As in analysis, the negative-frequency bands follow by conjugate symmetry, and the DC and Nyquist sidebands are made real, so that $\hat{x}$ is real. For analysed coefficients, substitution of (2) gives $\hat{x} = x$, so $ST = I$ on real signals in exact arithmetic.

3. PACKED ROUTING AND ARITHMETIC STRUCTURE

Because x is real, its spectrum $X = F_N x$ satisfies $X_{N-k} = \overline{X_k}$ for $1 \leq k < N$, so the bins $0 \leq k \leq N/2$ determine the whole spectrum. A full complex FFT does not exploit this redundancy. A classical real-input construction [26, 27] halves the transform length. Let $L = N/2$ and pack consecutive real samples into complex pairs for $0 \leq j < L$,

$$z_j = x_{2j} + i x_{2j+1}. \quad (5)$$

The packed spectrum $Z = F_L z$ holds exactly N real numbers, as many as the signal, and loses no information. Any required bin of X follows from two packed bins [15],

$$X_k = \frac{1}{2}(1 - i\omega_k) Z_{k \bmod L} + \frac{1}{2}(1 + i\omega_k) \overline{Z_{-k \bmod L}}, \quad (6)$$

where $\omega_k = e^{-2\pi i k / N}$ and packed indices wrap modulo L .

Substituting (6) into (3) gives each band spectrum directly from Z . If entry m of u_λ holds bin k ,

$$[u_\lambda]_m = \frac{1}{2} g_{\lambda,k} [(1 - i\omega_k) Z_{k \bmod L} + (1 + i\omega_k) \overline{Z_{-k \bmod L}}], \quad (7)$$

entries m that hold no bin k are zero. We call the map $R_\lambda: Z \mapsto u_\lambda$ the *router* of band λ . It combines bin recovery, windowing and selection in one table without forming X . Conjugation makes the router real-linear, though generally not complex-linear. With P denoting the packing in (5), the analysis of band λ factors exactly as

$$c_\lambda = T_\lambda x = F_{M_\lambda}^{-1} R_\lambda F_L P x. \quad (8)$$

Synthesis runs in the opposite direction. Inverting (6) gives each packed bin from two bins of the half-spectrum,

$$Z_q = \frac{1}{2} [(1 + i\overline{\omega_q}) X_q + (1 - i\overline{\omega_q}) \overline{X_{-q}}], \quad 0 \leq q < L, \quad (9)$$

where the conjugate term accounts for the negative frequencies. In (4), band λ contributes $X_k^{(\lambda)} = h_{\lambda,k} [E_\lambda^\top u_\lambda]_k$ for $0 \leq k \leq N/2$, made real at DC and Nyquist. Substituting $X^{(\lambda)}$ into (9) gives its packed spectrum $Z^{(\lambda)}$, and we call $\tilde{R}_\lambda: u_\lambda \mapsto Z^{(\lambda)}$ the *dual router* of band λ . With P^{-1} unpacking each complex sample into two real ones, synthesis is

$$S(\{c_\lambda\}_\lambda) = P^{-1} F_L^{-1} \sum_\lambda \tilde{R}_\lambda F_{M_\lambda} c_\lambda, \quad (10)$$

so the band contributions are added in the packed domain and a single inverse FFT of length L serves all bands.

3.1. Depth and block width

Analysis (8) has three stages, the packed transform F_L , routing R_λ , and band inverse transforms $F_{M_\lambda}^{-1}$; synthesis (10) reverses their order. Each stage splits into *layers* of small, independent linear blocks, which can be evaluated in parallel. We describe this structure independently of hardware by its *depth*, the number of layers that run in sequence, and its *block width*, the larger of a block's real input and output dimensions. Wiring between layers may reorder entries, change signs or insert zeros at no cost in this count, but may not duplicate inputs.

The Cooley–Tukey construction and its mixed-radix generalisations write a Fourier transform of length $n = r_1 \cdots r_d$ as d layers of small Fourier blocks, separated by permutations and twiddle factors [14, 15, 17]. The *radix* r_j is the length of a layer's Fourier blocks, so radix 2 combines pairs, radix 4 groups of four, and so on. If every radix is at most r , let $\tau_r(n)$ be the least number of factors in a product $n = r_1 \cdots r_d$ with $2 \leq r_j \leq r$. For example, $\tau_{32}(2^a) = \lceil a/5 \rceil$, so the packed transform of length $32,768 = 2^{15}$ needs three layers.

By (7), every entry of u_λ reads one pair of packed bins $\{Z_q, Z_{-q}\}$, namely when it holds bin q or bin $L - q$. Grouping entries by this unordered pair splits the router into independent blocks, each with at most four real inputs. We call the number of nonzero entries, over all bands, that read the pair $\{Z_q, Z_{-q}\}$ its *fan-out* D_q , and let $D = \max_q D_q$, so that no router block has more than $2D$ real outputs.

These counts give the following bound.

Theorem 1 (Ragged Fourier factorisation). *If L and every M_λ factor into integers between 2 and r , analysis and synthesis can be evaluated with depth at most*

$$d = \underbrace{\tau_r(L)}_{\text{packed FFT}} + \underbrace{1}_{\text{router}} + \underbrace{\max_\lambda \tau_r(M_\lambda)}_{\text{band FFTs}}, \quad (11)$$

and real block width at most

$$w = \max \left\{ \underbrace{2r}_{\text{FFT block}}, \underbrace{4}_{\text{router in}}, \underbrace{2D}_{\text{router out}} \right\}. \quad (12)$$

Proof. Cooley–Tukey gives the $\tau_r(L)$ and $\tau_r(M_\lambda)$ Fourier layers, with twiddle factors absorbed into the small Fourier blocks of at most $2r$ real values [14, 15]. Grouped by packed pair as above, the router consists of independent blocks with at most four real inputs and $2D$ real outputs. The router and dual router each occupy one layer, since synthesis uses the same pairs with reversed dimensions. Identity layers align shorter band plans without zero-padding their coefficient vectors. $\square$

3.2. Adjoints for training

Backpropagation through analysis and synthesis uses their adjoints. Write S_λ for band λ ’s contribution in (10), so $S(c) = \sum_\lambda S_\lambda c_\lambda$. Gradients use the real inner product $\langle u, v \rangle_{\mathbb{R}} = \text{Re}(u^H v)$ on complex arrays, where Re takes the real part. Write $\dagger$ for its adjoint, which moves a linear map from one side of this inner product to the other. Packing preserves the inner product, so $P^\dagger = P^{-1}$. Because of its conjugate term, the router has the form $Rz = Az + B\bar{z}$ for complex matrices A and B , and its real adjoint is $R^\dagger y = A^H y + B^T \bar{y}$.

Reversing the order of the factors and taking their real adjoints gives

$$T_\lambda^\dagger = \frac{L}{M_\lambda} P^{-1} F_L^{-1} R_\lambda^\dagger F_{M_\lambda}, \quad (13)$$

$$S_\lambda^\dagger = \frac{M_\lambda}{L} F_{M_\lambda}^{-1} \tilde{R}_\lambda^\dagger F_L P. \quad (14)$$

Thus $T_\lambda^\dagger$ has the form of synthesis with $R_\lambda^\dagger$ in place of $\tilde{R}_\lambda$, and $S_\lambda^\dagger$ the form of analysis with $\tilde{R}_\lambda^\dagger$ in place of R_λ . Hence $R_\lambda^\dagger$ reads the same pairs as the dual router, with $g_{\lambda,k}$ instead of $h_{\lambda,k}$ and twice the weight at DC and Nyquist, so backpropagation reuses the same index tables and Fourier routines. Because these maps are fixed and linear, no transform activations need be saved.

4. IMPLEMENTATION

The CUDA implementation¹ uses FP32 arithmetic. Table 1 gives the principal configuration, for a slice of $N = 65536$ and 44.1 kHz, which follows the multi-resolution layout of [28]. It combines three CQT resolutions, covering 3, 4, and 2 octaves at 8, 16, and 32 bands per octave. The 152 positive-frequency bands share one coefficient length per octave, obtained by rounding its largest window support up

¹<https://github.com/cucuwritescode/Flash-CQT>

Table 1. Frequency band layout at $N = 65,536$ and 44.1 kHz. DC and Nyquist sidebands have lengths 128 and 2048.

Octave	Bands	Centres (Hz)	M_λ	Radix plan
1	8	43–81	32	32
2	8	88–165	64	8×8
3	8	180–337	128	16×8
4	16	345–666	128	16×8
5	16	695–1342	256	16×16
6	16	1402–2708	512	32×16
7	16	2830–5464	1024	32×32
8	32	5512–10905	1024	32×32
9	32	11147–21810	2048	$32 \times 8 \times 8$

to a power of two. Band centres are rounded to Fourier bins, with the highest centre adjusted towards Nyquist. DC and Nyquist sidebands complete the coverage.

The fast kernels are specialised to a slice of $N = 65,536$, for which the packed length $L = 32,768 = 32^3$ gives three radix-32 layers with fixed memory offsets. The band kernels read routing tables and the radix plans of Table 1 at run time, but also assume this packed length. Packing is performed within the first layer, and the last writes interleaved real and imaginary values in the order in which the FFT computes them. The routing tables index that order directly [17].

One CUDA thread block of 256 threads handles each band and batch element, using at most 16.25 KiB of on-chip shared memory, including the precomputed Fourier weights. The index maps avoid write conflicts and permit stages to overwrite their inputs within one buffer. Synthesis evaluates (10), using band FFTs and atomic FP32 additions to accumulate dual-routed contributions before the packed inverse FFT.

Routing records are stored in the order in which the threads read them. If entry m of u_λ holds bin k , its 16-byte record holds two 16-bit indices into Z , the window value $g = g_{\lambda,k}$, and the complex weight $\alpha = \frac{1}{2}g(1 - i\omega_k)$ from (7). Recovering the other weight as $\frac{1}{2}g(1 + i\omega_k) = g - \alpha$ reduces table size.

For this layout, the fan-out is $D = 7$, attained at packed pairs 59 through 63. Theorem 1 gives depth $3 + 1 + 3 = 7$ and real block width 64 when $r = 32$. Under this radix cap, seven layers are necessary when Fourier-stage wiring preserves complex entries and the construction retains the packed spectrum Z , the band spectra u_λ , and a separate router layer. Indeed, d radix-32 layers let an output depend on at most 32^d complex inputs. Every output of the packed DFT depends on all 32,768 inputs, and every output of a length-2048 band inverse DFT depends on its 1,411 nonzero inputs. Each exceeds 32^2 and requires three layers; the router requires one more. This bound concerns the stated arithmetic structure, not runtime optimality or GPU kernel launches.

Signals shorter than the slice length $N = 65,536$ samples require omitting the lowest bands or lowering Q , which shortens their windows at the cost of frequency selectivity, and would need new specialised kernels. For signals longer than $N = 65,536$, we transform slices of N samples independently, with 50% overlap and a Tukey transition window [10].

5. EVALUATION

Measurements use the configuration in Table 1, with $N = 65536$, FP32 arithmetic, and inputs and tables resident on the GPU. The

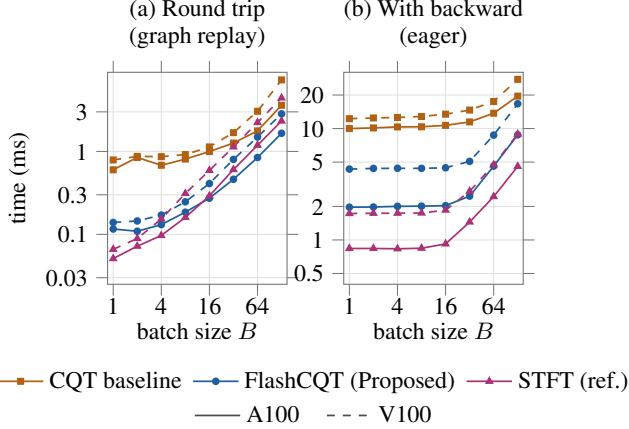


Fig. 2. Time against batch size on A100 (solid) and V100 (dashed). (a) Round trip (analysis and synthesis) under CUDA graph replay. (b) Round trip with backward pass (analysis, synthesis, squared loss and backpropagation) in eager mode.

devices are an NVIDIA A100-SXM4 with 80 GB and a V100-SXM2 with 32 GB, both using PyTorch 2.12.0 and CUDA 12.6. CUDA events give median times over 50 runs after three warm-ups; the text quotes A100 results unless stated otherwise.

The *CQT baseline* evaluates the same operator in PyTorch using cuFFT-backed calls. It computes a real FFT, gathers and weights the bands using precomputed tables, and batches equal-length band transforms by octave. The inverse weights and combines their spectra before one real inverse FFT. Both methods retain the same windows, native coefficient lengths, and sidebands, without zero-padding to a common length. Differences in time and memory therefore come from the implementation, not from the operator.

The STFT provides a reference for the cost of a common transform with nearly the same number of coefficients on the same device. With a 2048-sample Hann window, hops of 512 samples, and centred framing, it produces 132,225 complex coefficients, within 0.6% of the CQT’s 132,992, while using uniform frequency resolution. Eager measurements use `torch.stft` and `torch.istft`. Under graph replay, the inverse is an overlap-add with precomputed window normalisation, which agrees with `torch.istft` above 120 dB.

5.1. Analysis and reconstruction

Each round trip is captured as a CUDA graph, in Fig. 2, which records its kernel sequence for inputs of fixed shape, and timed under replay. This avoids Python dispatch between kernels and measures the GPU execution that the factorisation targets. Flash-CQT is 5.2 times faster than the CQT baseline at $B = 1$, and across batch sizes B the speedup ranges from 2.1 to nearly 8 on A100 and 2.0 to 6.0 on V100 (Fig. 2(a)). Under graph replay, the proposed CQT takes 2.3 times the STFT time at $B = 1$ and reaches parity near $B = 16$. This comparison depends on the inverse implementation, since at $B = 128$ the native eager STFT round trip takes 1.39 ms, below the proposed CQT’s 1.66 ms and the graph STFT’s 2.32 ms.

Table 2 reports reconstruction Signal-to-Noise Ratio (SNR), defined as $10 \log_{10}(\|x\|_2^2 / \|x - STx\|_2^2)$, as the median over batch sizes. Flash-CQT gives 129.8 dB, varying by at most 0.1 dB, or about 3×10^{-7} relative error in the Euclidean norm. The measured error is a few times the FP32 machine epsilon of 1.2×10^{-7} and is consistent with round-off; the exact-arithmetic identity $ST = I$ does not imply

Table 2. Reconstruction SNR, workspace per signal and $B=1$ timings on an NVIDIA A100. Round trips use graph replay; steps include eager backpropagation. Best values among the CQT implementations are bold.

Method	Multi-res.	SNR (dB)	Workspace (MB)	Round trip (ms)	Step (ms)
STFT (reference)	✗	133.1	4.3	0.051	0.838
CQT baseline	✓	126.8	4.2	0.601	10.003
Flash-CQT (proposed)	✓	129.8	2.6	0.116	1.969

bitwise reconstruction. Transient workspace is the peak extra allocation during a call, excluding inputs and persistent tables, reported per signal because it grows linearly with B . Keeping the spectrum packed and overwriting buffers in place reduces this workspace by 36 to 38% across batches.

5.2. Differentiable round trip

The second experiment adds squared error against a target and back-propagation through both transforms to measure a differentiable round trip without a neural network. It runs in PyTorch eager mode, so its times include host dispatch and autograd overhead. The baselines use PyTorch autograd, while Flash-CQT uses the adjoints in Section 3.2. Analysis and synthesis gradients agree with autograd at 130.05 and 129.48 dB, respectively.

Eager speedups over the CQT baseline range from 2.2 to 5.3 on A100 and 1.7 to 3.0 on V100 (Fig. 2(b)). In the baseline, overlapping windows make the backward pass of the band gather a scatter-add with repeated indices, whereas the explicit adjoints reuse the Fourier stages and routing tables. Relative to the STFT, the CQT baseline costs 12 times as much at $B = 1$ and 4.3 times at $B = 128$, whereas FlashCQT costs 2.35 and 1.9 times as much. Effects on complete model training and comparison with a matched cuFFTDx implementation remain open.

6. CONCLUSION

This paper presented Flash-CQT, an exact factorisation of the invertible CQT for efficient GPU computation. This factorisation combines the operations between the Fourier stages without changing the invertible CQT. Fixed routing recovers and weights each band’s bins inside its transform kernel, preserving native lengths, while the factors give reconstruction, real adjoints, and arithmetic bounds. On the tested GPUs, this organisation reduces round-trip time by factors of two to eight against the CQT baseline, and uses over a third less temporary workspace while retaining about 130 dB reconstruction SNR.

These savings matter most where the transform is applied repeatedly, as in the training and inference of neural audio models. Applications that require invertible CQT coefficients, such as diffusion-based audio generation [28], music source separation [13], audio bandwidth extension [29, 30] and timbre transfer [31], can therefore use the transform within a small factor of the STFT cost while retaining its logarithmic frequency resolution. CQT-based training losses and discriminators [32–34] can likewise benefit from the fused analysis kernels even without invertibility, and from the exact adjoints of Sec. 3.2 for their gradients. The construction also applies to other painless NSGTs beyond the CQT configuration evaluated in this paper, such as auditory filterbanks [35], although their fan-out and performance remain to be established. Slicing further extends the implementation to long recordings and streaming.

7. ACKNOWLEDGEMENTS

We acknowledge the computational resources provided by the Aalto Science-IT project. This work was supported by the HUCE infrastructure of the Aalto School of Electrical Engineering. This research was funded in part by the Research Council of Finland (grant no. 371845).

8. REFERENCES

- [1] J. O. Smith, *Spectral Audio Signal Processing*, W3K Publishing, 2011, Online book, 2011 ed. [Online]. Available: <https://ccrma.stanford.edu/~jos/sasp/>.
- [2] H. Purwins, B. Li, T. Virtanen, et al., “Deep learning for audio signal processing,” *IEEE J. Sel. Topics Signal Process.*, vol. 13, no. 2, pp. 206–219, 2019.
- [3] J. B. Allen and L. R. Rabiner, “A unified approach to short-time Fourier analysis and synthesis,” *Proc. IEEE*, vol. 65, no. 11, pp. 1558–1564, 1977.
- [4] J. C. Brown, “Calculation of a constant Q spectral transform,” *J. Acoust. Soc. Am.*, vol. 89, no. 1, pp. 425–434, 1991.
- [5] B. R. Glasberg and B. C. J. Moore, “Derivation of auditory filter shapes from notched-noise data,” *Hearing Res.*, vol. 47, no. 1–2, pp. 103–138, 1990.
- [6] M. Müller, *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*, Springer, 2015.
- [7] C. Schörkhuber and A. Klapuri, “Constant- Q transform toolbox for music processing,” in *Proc. Sound and Music Comput. Conf.*, Barcelona, Spain, 2010.
- [8] P. Balazs, M. Dörfler, F. Jalliet, et al., “Theory, implementation and applications of nonstationary Gabor frames,” *J. Comput. Appl. Math.*, vol. 236, no. 6, pp. 1481–1496, 2011.
- [9] G. A. Velasco, N. Holighaus, M. Dörfler, et al., “Constructing an invertible constant- Q transform with nonstationary Gabor frames,” in *Proc. Int. Conf. Digital Audio Effects*, Paris, France, 2011, pp. 93–99.
- [10] N. Holighaus, M. Dörfler, G. A. Velasco, et al., “A framework for invertible, real-time constant- Q transforms,” *IEEE Trans. Audio Speech Lang. Process.*, vol. 21, no. 4, pp. 775–785, 2013.
- [11] K. W. Cheuk, H. Anderson, K. Agres, et al., “nnAudio: An on-the-fly GPU audio to spectrogram conversion toolbox using 1D convolutional neural networks,” *IEEE Access*, vol. 8, pp. 161981–162003, 2020.
- [12] E. Moliner, “CQT_pytorch: PyTorch implementation of the invertible CQT based on nonstationary Gabor filters,” [Online]. Available: https://github.com/elaimoliner/CQT_pytorch, Accessed: Sep. 12, 2026.
- [13] S. Hanssian, “Music demixing with the sliCQ transform,” in *Proc. Music Demixing Workshop (MDX21), satellite event of ISMIR*, 2021.
- [14] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.
- [15] C. Van Loan, *Computational Frameworks for the Fast Fourier Transform*, SIAM, Philadelphia, PA, USA, 1992.
- [16] W. M. Gentleman and G. Sande, “Fast Fourier transforms—for fun and profit,” in *Proc. AFIPS Fall Joint Comput. Conf.*, 1966, vol. 29, pp. 563–578.
- [17] C. Temperton, “Self-sorting mixed-radix fast Fourier transforms,” Tech. Memo. 66, ECMWF, 1982.
- [18] J. Johnson, R. W. Johnson, D. Rodriguez, et al., “A methodology for designing, modifying, and implementing Fourier transform algorithms on various architectures,” *Circuits Syst. Signal Process.*, vol. 9, no. 4, pp. 449–500, 1990.
- [19] D. H. Bailey, “FFTs in external or hierarchical memory,” *J. Supercomput.*, vol. 4, no. 1, pp. 23–35, 1990.
- [20] A. Edelman, P. McCorquodale, and S. Toledo, “The future fast Fourier transform?,” *SIAM J. Sci. Comput.*, vol. 20, no. 3, pp. 1094–1114, 1999.
- [21] T. Dao, B. Chen, N. S. Sohoni, et al., “Monarch: Expressive structured matrices for efficient and accurate training,” in *Proc. ICML*, 2022, pp. 4690–4721.
- [22] D. Y. Fu, H. Kumbong, E. Nguyen, et al., “FlashFFTConv: Efficient convolutions for long sequences with tensor cores,” in *Proc. ICLR*, 2024.
- [23] S. Wu, Y. Zhai, H. Dai, et al., “TurboFNO: High-performance Fourier neural operator with fused FFT-GEMM-iFFT on GPU,” in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2025.
- [24] NVIDIA, “cuFFTDx documentation,” <https://docs.nvidia.com/cuda/cufftdx/>, Accessed: Sep. 11, 2026.
- [25] I. Daubechies, A. Grossmann, and Y. Meyer, “Painless nonorthogonal expansions,” *J. Math. Phys.*, vol. 27, no. 5, pp. 1271–1283, 1986.
- [26] H. V. Sorensen, D. L. Jones, M. T. Heideman, et al., “Real-valued fast Fourier transform algorithms,” *IEEE Trans. Acoust. Speech Signal Process.*, vol. 35, no. 6, pp. 849–863, 1987.
- [27] C. Temperton, “Fast mixed-radix real Fourier transforms,” Tech. Memo. 71, ECMWF, 1983.
- [28] M. V. M. da Costa and E. Moliner, “An octave-based multi-resolution CQT architecture for diffusion-based audio generation,” in *Proc. IEEE Int. Symp. Internet of Sounds (IS2)*, L’Aquila, Italy, 2025, pp. 1–7.
- [29] Eloi Moliner, Filip Elvander, and Vesa Välimäki, “Blind audio bandwidth extension: A diffusion-based zero-shot approach,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 5092–5105, 2024.
- [30] M. Ali, M. C. Oener, A. Bashir, et al., “Exploiting constant- q transform and its variant in light-weight neural network framework for artificial bandwidth extension,” in *Proc. Eur. Signal Process. Conf. IEEE*, 2025, pp. 296–300.
- [31] S. Huang, Q. Li, C. Anil, X. Bao, S. Oore, and R. B. Grosse, “TimbreTron: A WaveNet(CycleGAN(CQT(Audio))) pipeline for musical timbre transfer,” in *Proc. ICLR*, 2019.
- [32] Y. Li, R. Yuan, G. Zhang, et al., “MERT: Acoustic music understanding model with large-scale self-supervised training,” in *Proc. ICLR*, 2024.
- [33] S. Welker, M. Le, R. T. Q. Chen, W.-N. Hsu, T. Gerkmann, A. Richard, and Y.-C. Wu, “FlowDec: A flow-based full-band general audio codec with high perceptual quality,” in *Proc. ICLR*, 2025.
- [34] Y. Gu, X. Zhang, L. Xue, and Z. Wu, “Multi-scale sub-band constant- Q transform discriminator for high-fidelity vocoder,” in *Proc. Int. Conf. Acoust. Speech Signal Process.*, 2024, pp. 10616–10620.
- [35] T. Necciari, P. Balazs, N. Holighaus, et al., “The ERBlet transform: An auditory-based time-frequency representation with perfect reconstruction,” in *Proc. Int. Conf. Acoust. Speech Signal Process.*, Vancouver, BC, Canada, 2013, pp. 498–502.